

EVA Build Plan — v0.3

Research Node

Stage 1 output — Architect Agent

The One Rule applies throughout: do not redesign

EVA, improve implementation only.

Where We Actually Are

This is not a from-scratch build. EVA already has a partially working implementation:

-  `eva/api.py` — complete, working HTTP API (stdlib `http.server`, no external deps)
-  `eva/tools/` — `__init__.py`, `base.py`, `web_search.py` (DuckDuckGo, no API key), `gateway.py` (Sentinel-gated execution) — existence confirmed, contents not yet reviewed
-  A working frontend dashboard (single-file HTML/JS, no build step) — confirmed and functioning against the real API surface
-  `eva/core.py` and `eva/models.py` — referenced everywhere, contents not yet seen
-  Architecture intent (agent roles, table list, non-goals) — documented in the v0.3 Engineering

Handoff Package

The handoff doc's *recommended* stack (FastAPI/PostgreSQL/pgvector/LangGraph/React/Docker) is **aspirational and not what's built**. Real code already deviated from it for good, documented reasons (no network access in the build sandbox). Stage 2 onward should build on what exists, not revert to the handoff doc's wishlist.

Sequencing for Remaining Stages

Before Stage 2 can safely start

Close the four open questions logged in

`EVA_ENGINEERING_SPEC.md` :

1. Contents of `core.py` (DB engine, EVACore wiring, how Pixie/Nova connect to Sentinel/Ghost)
2. Contents of `models.py` (full enum values, field definitions)
3. Contents of `tools/gateway.py` (how Sentinel-gating is actually enforced)
4. Full `/tool/execute` + `/tool/list` route bodies and any CORS handling in `api.py`

Recommendation: don't hand off to Stage 2 (Database/Backend Agent) until at least `core.py` is reviewed — otherwise Stage 2 risks writing a

schema/migration that conflicts with what's already running.

Stage 2 — Database/Backend Agent

- Confirm real DB engine before writing any migration
- Reconcile the 8-table list from the handoff doc (`users`, `missions`, `tasks`, `agents`, `memory_records`, `sources`, `permissions`, `audit_logs`) against what `core.py` actually persists
- Do not introduce PostgreSQL/pgvector unless `core.py` shows it's already there — if it's SQLite, stay on SQLite

Stage 3 — AI Agent Builder

(Pixie/Nova/Sentinel)

- Ghost and Sentinel already have live behavior via `core.ghost` / `core.sentinel` — this stage should extend/harden what exists, not rewrite it
- Pixie is referenced conceptually (handoff doc) but has no confirmed code yet — needs an implementation pass once `core.py` clarifies whether it's a module or logic inside `EVACore`
- Nova's tool use goes through the Tool Gateway (`/tool/execute`) — confirmed pattern to build on

Stage 4 — Security Review Agent

- Focus areas once code is fully visible: Sentinel's authorization logic (`review_task`), the Tool Gateway's enforcement of Sentinel approval, and the currently-unconfirmed CORS setup (a real gap if the frontend calls a permissive/open API cross-origin)

Stage 5 — Coding Agent

- Repo cleanup against the confirmed file list (`api.py` , `tools/*`) plus whatever `core.py` / `models.py` reveal
- Do not scaffold the handoff doc's proposed directory tree (`app/core/` , `app/agents/` , etc.) unless it matches what's real — current evidence suggests a flatter `eva/` package, not the deeper `app/` structure the doc proposes

Non-Goals (carried forward, still binding)

No sentience claims, no unrestricted autonomy, no self-modification, no consciousness layers, no large multi-agent swarms.

Immediate Next Action

Get `eva/core.py` and `eva/models.py` into this pipeline. They're the load-bearing files every other document defers to — everything marked ? in the three companion documents traces back to one of these two files.